

Android 广播及替代方案对比

Android广播（Broadcast）详解

一、广播的基本概念

广播是Android中用于组件间通信的机制，允许应用发送/接收系统或自定义事件通知。它基于发布-订阅模式，发送方（广播者）无需知道接收方（广播接收器）的存在，接收方通过注册监听特定广播类型来响应事件。

二、广播的分类

根据发送方式和权限，广播分为以下类型：

类型	特点
标准广播（Normal）	异步传播，所有接收器同时接收，无法截断
有序广播（Ordered）	同步传播，按优先级接收，可截断或修改数据
本地广播（Local）	仅应用内传播，更安全高效
系统广播	系统事件触发（如开机、网络变化），需声明权限

三、广播接收器（BroadcastReceiver）的使用

1. 静态注册（AndroidManifest.xml）

适合常驻监听（如开机启动），但Android 8.0后对隐式广播限制严格。

Code block

```
1 <receiver
2     android:name=".MyReceiver"
3     android:exported="true">
4     <intent-filter>
5         <!-- 监听网络变化 -->
6         <action android:name="android.net.conn.CONNECTIVITY_CHANGE" />
7         <!-- 自定义广播 -->
8         <action android:name="com.example.MY_BROADCAST" />
9     </intent-filter>
```

```
10 </receiver>
```

2. 动态注册（代码中）

灵活控制生命周期，需在 `onDestroy` 中注销。

Code block

```
1  public class MainActivity extends AppCompatActivity {
2      private MyReceiver receiver;
3
4      @Override
5      protected void onCreate(Bundle savedInstanceState) {
6          super.onCreate(savedInstanceState);
7          setContentView(R.layout.activity_main);
8
9          receiver = new MyReceiver();
10         IntentFilter filter = new IntentFilter();
11         filter.addAction("android.net.conn.CONNECTIVITY_CHANGE");
12         registerReceiver(receiver, filter); // 注册
13     }
14
15     @Override
16     protected void onDestroy() {
17         super.onDestroy();
18         unregisterReceiver(receiver); // 注销
19     }
20 }
```

3. 广播接收器实现

Code block

```
1  public class MyReceiver extends BroadcastReceiver {
2      @Override
3      public void onReceive(Context context, Intent intent) {
4          String action = intent.getAction();
5          if ("android.net.conn.CONNECTIVITY_CHANGE".equals(action)) {
6              // 处理网络变化逻辑
7              Toast.makeText(context, "网络状态改变", Toast.LENGTH_SHORT).show();
8          }
9      }
10 }
```

四、发送广播

1. 发送标准广播

Code block

```
1 Intent intent = new Intent("com.example.MY_BROADCAST");
2 sendBroadcast(intent);
```

2. 发送有序广播（可设置权限和优先级）

Code block

```
1 Intent intent = new Intent("com.example.ORDERED_BROADCAST");
2 sendOrderedBroadcast(intent, null); // 第二个参数为权限
```

3. 本地广播（LocalBroadcastManager）

Code block

```
1 LocalBroadcastManager manager = LocalBroadcastManager.getInstance(this);
2 Intent intent = new Intent("com.example.LOCAL_BROADCAST");
3 manager.sendBroadcast(intent);
```

五、权限与安全性

1. 发送权限：发送广播时指定权限，接收器需声明对应权限才能接收。

Code block

```
1 sendBroadcast(intent, "com.example.PERMISSION");
```

2. 接收权限：在Manifest中声明权限：

Code block

```
1 <uses-permission android:name="com.example.PERMISSION" />
```

3. Android 8.0+限制：隐式广播需动态注册，或使用 `setComponent` 指定组件。

六、第三方替代方案

原生广播存在跨应用安全风险、效率低等问题，以下是常用替代方案：

1. EventBus（GreenRobot）

- **原理：**基于观察者模式，简化组件通信。

- **优势：**解耦、高效、支持线程切换。

使用步骤：

1. 添加依赖：

Code block

```
1 implementation 'org.greenrobot:eventbus:3.3.1'
```

2. 定义事件类：

Code block

```
1 public class MessageEvent {
2     private String message;
3     // getter/setter
4 }
```

3. 注册/注销订阅者：

Code block

```
1 @Override
2 protected void onStart() {
3     super.onStart();
4     EventBus.getDefault().register(this);
5 }
6
7 @Override
8 protected void onStop() {
9     super.onStop();
10    EventBus.getDefault().unregister(this);
11 }
```

4. 接收事件：

Code block

```
1 @Subscribe(threadMode = ThreadMode.MAIN)
2 public void onMessageEvent(MessageEvent event) {
3     Toast.makeText(this, event.getMessage(), Toast.LENGTH_SHORT).show();
4 }
```

5. 发送事件：

Code block

```
1 EventBus.getDefault().post(new MessageEvent("Hello EventBus!"));
```

2. RxBus (基于RxJava)

- **原理**：通过RxJava的Subject实现事件总线。
- **优势**：结合RxJava操作符，灵活处理异步流。

示例：

Code block

```
1 public class RxBus {
2     private static volatile RxBus instance;
3     private final Subject<Object> bus = PublishSubject.create().toSerialized();
4
5     private RxBus() {}
6
7     public static RxBus getInstance() {
8         if (instance == null) {
9             synchronized (RxBus.class) {
10                 if (instance == null) {
11                     instance = new RxBus();
12                 }
13             }
14         }
15         return instance;
16     }
17
18     public void post(Object event) {
19         bus.onNext(event);
20     }
21
22     public <T> Observable<T> toObservable(Class<T> eventType) {
23         return bus.ofType(eventType);
24     }
25 }
```

3. Otto (Square)

- **原理**：简化版EventBus，专注Android组件通信。
- **注意**：已停止维护，推荐使用EventBus替代。

七、广播与替代方案对比

方案	适用场景	优势	劣势
原生广播	系统事件、跨应用通信	系统级支持	效率低、安全风险
EventBus	应用内组件通信	高效、解耦、线程控制	需引入第三方库
RxBus	复杂异步流处理	结合RxJava生态	学习成本高

八、总结

- **原生广播**：适合系统事件监听或跨应用通信，但需注意权限和版本限制。
- **第三方事件总线（EventBus/RxBus）**：更适合应用内组件通信，简化代码结构，提升开发效率。
- **选择方案时需根据场景权衡**：系统级事件用广播，应用内通信优先EventBus。

（注：文档部分内容可能由 AI 生成）